OpsDesk for TBO(ZenDesk)

Build a next-generation operations ticketing platform shaped by TBO's real B2B travel complexity.

Relationships, context, and urgency first — isolated tickets second.

Document part	What it covers
Part A — The Problem	Why today's ticketing system falls short for TBO
Part B — The Challenge	What to build, demo, and how you'll be judged

---

Part A — The Problem

Background

The current ticketing system was designed for a world where **speed and efficiency** were the only metrics that mattered. It breaks customer interactions into isolated tasks — **tickets** — that focus on **resolution rather than relationships**.

That model feels increasingly out of place for TBO's ever-changing **B2B** landscape. We expect:

- Personalized experiences
- Proactive solutions
- A seamless journey across contacts

Instead, the fragmented and reactive nature of the current system leaves only two expensive paths:

Option	What it means
A	Incrementally pay for each additional service
B	Integrate additional TPS APIs — and keep paying for the glue

Core question

The problem is **not** whether the ticketing system can manage tickets.
It is whether the current solution can support **TBO's operational complexity efficiently and economically**.

Problem snapshot

#	Problem	In one line
1	Tickets without context	No history, intent, or ticket linkage
2	Routing & ownership	Hardcoded FIFO/LIFO; weak time/region/ranking logic
3	Side conversations	Recipients stuck to group; break on transfer
4	SLA & escalation	Travel urgency hard; no direct escalate to EMs
5	Broken feedback loop	Insights trapped; AI costs extra + needs HITL
6	Data & archival	3K-row exports; no realtime; API/storage tax

---

1. Tickets without context

Every interaction is treated as an **isolated task**. There is no continuity — and little understanding of the customer's **history** or **intent**.

Gap	Detail
Agency intent	No reliable classification. Subject/text triggers are manually created and error-prone.
Ticket linkage	Related issues become separate tickets unless the agency replies on the same thread.
Agent visibility	No view of previous interactions → frustration builds → churn risk.

Example

An agency contacts support about the **same refund three times**. Each time, the ticketing system treats it as a **brand-new problem** and opens a **separate ticket** — unless they stay on the same thread.

Makeshift today

A **third-party app** on the ticketing platform surfaces tickets that share the same **TBOH**. Linkage should be a **first-class** product capability — not a bolt-on.

---

2. Inefficient ticket routing and ownership

Limitation	What happens today
Hardcoded triggers	Rule-based assignment leaves little room for overlapping or advanced routing.
FIFO / LIFO	Default queue order; customization is mostly by agency **country ID**.

Post-routing moves	Triggers can move tickets after the fact — still a **unilateral** flow.
Missing logic	Practical **to-and-from** routing by **time**, **priority**, and **availability** is not possible.

What we need routing to consider: **time-based · region-based · ranking-based · availability**.

---

3. Side conversations — supplier / agency

Side conversations with suppliers or agencies are fragile.

Issue	Impact
Recipient list **can't be set** freely	Always defaults to the ticket's **group**
Ticket is **transferred**	Recipients often **stay the same**
Result	Delivery errors and broken threads

---

4. SLA and escalation complexity

Travel ops need SLAs driven by real-world urgency — not a one-size-fits-all clock.

Drivers that matter:

- Travel urgency
- Check-in proximity
- LCD
- Regional calendars
- Dependencies between teams

Need	Reality today
Prioritise by **check-in date** or **LCD**	Requires heavy / incomplete configuration
Escalate directly to **Escalation Managers**	Not available — only "add followers" or email triggers

---

5. A broken feedback loop

Agency conversations hold a **goldmine**: recurring issues, special requests, and bug signals.

Symptom	Consequence
Insights stay locked in tickets	Product roadmaps disconnect from customer needs

AI / Copilot-style tools	Require **incremental spend**
Context still incomplete	Often needs a **human in the loop** to link conversations manually

---

6. Real-time data limits & archiving at a cost

Pain	Detail
Export caps	**~3K rows** at a time vs commitments closer to **50K**
No realtime sync	Weak for governance and live ops dashboards
API limits	Limits on ticket updates; step-ups charged incrementally
Archival	Storage / archival comes at an extra cost

Part B — The Hackathon Challenge

Vision

Design and build **OpsDesk** — a TBO-shaped ops ticketing MVP that proves we can do better than isolated, reactive tickets: **efficiently and economically**.

From (today)	To (OpsDesk)
Isolated tickets, no history / intent	Cases linked by agency, TBOH, intent, and history
Manual, error-prone classification	Structured agency-intent classification
Makeshift TBOH third-party app	First-class ticket linkage
Hardcoded FIFO / LIFO triggers	Routing by region, time, ranking, priority, availability
Side-convo recipients stuck to group	Explicit recipients that survive transfer
Generic SLAs; escalate via followers / email	Travel-aware SLAs + direct escalate to Escalation Managers
Insights locked in tickets; paid AI + HITL	Lightweight insight loop from conversations
3K-row exports; no realtime; API / archive tax	Queryable ops data path (document how you'd scale)

You are **not** expected to replace an entire enterprise ticketing suite in one weekend.
Ship a coherent vertical slice.

Target users

Persona	Primary needs
Agency / Requester	Continuity across contacts · status visibility · fewer repeat explanations
Ops Agent	History + linked tickets + intent · correct routing · clean supplier/agency threads
Escalation Manager	Direct escalate path · check-in / LCD urgency visibility
Ops Lead / Governance	Queue health · SLA risk · reporting without painful row/API limits
Product / Insights	Recurring themes from conversations → backlog (not lost in noise)

Scope assumption: single TBO organization for the hackathon (multi-tenant = optional stretch).

Must-have MVP features

Each feature maps to a problem in Part A.

MVP-1 · Context-rich tickets → Problem 1

- Create tickets with: title, description, agency identity, **TBOH**, priority, category / **intent**, and travel fields (check-in / LCD).
- **Link** tickets by TBOH, agency, and/or intent so agents see prior contacts (no third-party app in the demo).
- **Intent taxonomy:** manual select + simple auto-suggest (rules or lightweight ML — document your approach). Avoid free-text + brittle keywords only.
- Status flow example: `New` → `Open` → `Pending` → `Escalated` → `Resolved` → `Closed`
- Activity timeline: status, assignment, links, escalations.

MVP-2 · Routing & ownership → Problem 2

- Assign / reassign / unassign with clear ownership.
- Beyond FIFO / LIFO: support at least **two** of — region/country · time window · priority/ranking · agent availability (simulated OK).
- Views: triage (unassigned) · my queue · by region · by intent · linked-case view.
- Show **why** a ticket was routed (short audit reason is enough).

MVP-3 · Side conversations → Problem 3

- Side-conversation threads with an **explicit recipient list** (not bound to ticket group).
- Recipients stay correct after **transfer** / reassignment.
- Distinguish **internal notes** vs **external replies**.

MVP-4 · Travel-aware SLA & escalation → Problem 4

- SLA using **priority + check-in / LCD proximity** (optional region calendar flag).
- At-risk / breached indicators on the queue.
- **Escalate to Escalation Manager** as a first-class action (not only follower / email). Changes owner/queue; visible on timeline.

MVP-5 · Feedback / insight loop → Problem 5

- Tag or extract themes (e.g. refund delay, inventory sync, special request).
- **Insights** view: top themes this week + sample tickets.
- Optional: "suggested link to product / bug" field.
- Stretch: AI assist that uses **linked history** without manual stitching.

MVP-6 · Data access for ops → Problem 6

- Persist tickets (refresh must not wipe the queue).
- Export or API **not** artificially capped like a 3K-row UX at demo scale — document how you'd scale toward large extracts and near-realtime sync.
- Clear retention / archival stance for the demo.

MVP-7 · Roles & access

Role	Access
Requester (agency)	Own tickets
Agent	Org queues
Escalation Manager	Escalation queue + escalate actions
Lead	Lead views (may combine with Agent)

Demo auth is acceptable — document demo users. Do not expose other agencies' data.

Nice-to-have (stretch)

- Real-time dashboard sync (websocket or polling)
- Stronger auto-intent from conversation text
- Supplier portal thread UX
- Webhooks for ticket create / update
- CSAT after resolve
- Macros / canned replies with intent-aware suggestions

Out of scope (unless MVP is done)

- Full omnichannel (phone, WhatsApp, social)
- Full enterprise suite parity or production migration tooling
- Production SSO / billing marketplace

Demo journeys

Walk judges through these (or a clear subset — state what you cut).

Journey	Proves	Script
A — Context & linkage	Problem 1	Agency opens a refund ticket (TBOH). Agent sees **two prior tickets** on the same TBOH/intent. Intent is classified.
B — Routing	Problem 2	Urgent ticket routes by **region + priority + availability**. Agent sees routing reason. Ownership is clear.
C — Side conversation	Problem 3	Supplier side convo with **explicit recipients** → transfer ticket → recipients stay correct.
D — SLA & escalate	Problem 4	Near check-in/LCD → at-risk SLA → **escalate to Escalation Manager** → worked from escalation queue.
E — Insights & data	Problems 5 & 6	Lead opens Insights (themes) and exports/queries beyond a toy row limit.

Technical expectations

- Working **demo** over slides
- Stack + run steps documented in `README.md`
- Durable storage for tickets, links, comments, side conversations
- Empty / loading / error states on main screens
- Security basics: no hardcoded secrets · validate input · least privilege by role

Stack: no mandate — ship what your team finishes.

Deliverables

#	Deliverable
1	Runnable application (local or deployed URL)
2	README: setup, demo accounts, feature checklist **mapped to Problems 1–6**
3	3–5 min demo script (linkage · routing · side convo · escalate · insights)
4	Optional: architecture diagram + cost/complexity contrast vs “pay per add-on / TPS API”

Seed data (strongly recommended)

Seed	Why
3+ agencies; tickets **sharing TBOH** + repeat intents	Show “same refund, three contacts”
Mix of regions & priorities; 2–3 SLA at-risk / breached	Show check-in / LCD urgency

One **transferred** ticket with active supplier side convo	Prove recipient fix
Escalation Manager queue with 1–2 items	Prove first-class escalate
Theme tags for Insights	Prove feedback loop

Pitch lines

OpsDesk replaces isolated tickets from a traditional ticketing system with context-rich, travel-aware operations cases — linked by agency and TBOH, routed by reality, escalated on purpose.

The problem isn't that tickets can't be managed — it's whether the current ticketing solution can support **TBO's B2B complexity efficiently and economically**.

Assumptions to document

If organizers don't clarify in time, state these in your README:

- Demo login vs real IdP
- Simulated agent availability / regional calendars
- Intent model = rules vs lightweight ML
- Single-tenant TBO org for the hackathon

TBO Hackathon: AI-Native HR Services Platform(DarwinBox)

The challenge

Design and build an agent-first HR services platform that could replace a conventional HRMS. Agents do the routine work end to end, humans make the decisions that matter, and the platform runs at a low, measured cost per task.

Today, an HRMS records events but people still move the work. When a new hire is added, HR still emails IT for a laptop, Admin for an access card and Finance for payroll setup, then chases each one. Your platform should close those loops on its own.

You may demonstrate one HR service line in depth, such as talent acquisition, onboarding, leave, employee helpdesk or exits. The product itself must be a platform

that TBO's in-house developers can extend to new service lines once HR identifies and approves a need.

Objectives

Your submission must show all eight objectives. Depth on one service line beats shallow coverage of many.

1. **Automate end to end.** Agents trigger and complete cross-team work (IT, Admin, Finance, the hiring manager) without manual emails or follow-ups. Report what share of your chosen journey's steps finish with no human touch.
2. **Build a platform, not a point solution.** A TBO developer must be able to add a new service line, agent, workflow or integration without changing the core. Show a documented extension model: workflow definitions, an agent or skill registry, and API-first integrations.
3. **Keep humans on the decisions that matter.** Reserved decisions are routed to a named human approver and cannot be completed by an agent. The list of reserved decisions is configuration that HR can change, not hard-coded logic (see Guardrails).
4. **Comply by design.** Legal and policy rules sit in a governed layer that is versioned, testable and auditable, not buried in prompts.
5. **Keep a clean system of record.** Hold the employee lifecycle from hire to exit in a sound data model, with role-based access and a full audit trail of every change and who or what made it.
6. **Fail safely.** When an agent is unsure, a system is down or data conflicts, the work goes to a human queue with context attached. Retries must not duplicate actions, for example ordering two laptops.
7. **Serve people where they work.** Employees and managers should be able to ask and act through a conversational interface (web, Teams or Slack), not only through forms.
8. **Prove cost and impact with numbers.** Show cost per completed workflow, automation rate, cycle time and human-escalation rate on a simple dashboard (see Cost and model rule).

Guardrails (non-negotiable)

Automation stops at decisions that carry legal, financial or human consequences. A submission that lets an agent complete any of these on its own is disqualified from the top scores.



governed action flow · 7 steps, 1 decision

Every agent action passes a policy check. Reserved or low-confidence actions wait for a named human, and every outcome is logged.

Reserved for a human (minimum set; HR can add to it):

- Accepting a resignation, and agreeing a release date or notice-period buyout
- Final hiring decisions, offer approval and compensation
- Termination, disciplinary action and performance improvement plans
- Salary, grade or role changes, and any exception to policy
- Anything involving POSH, grievance or whistleblower complaints: route only, never decide
- Sign-off on statutory filings and payroll runs

Must be designed in:

- **Data protection:** comply with India's DPDP Act, 2023 and its Rules. That means purpose limitation, consent where needed, retention limits, and support for access and correction requests.
- **PII handling:** show exactly what personal data reaches a model and where it is hosted. Mask or minimise it before any external model call.
- **Fair hiring:** no automated rejection of a candidate without human review. Explain why each candidate was ranked where they were.
- **Explainability:** every agent action records its inputs, the rule or prompt version, the model used and its reasoning.
- **Access control:** role-based permissions, so an agent can never do more than the person it acts for.
- **Kill switch:** HR can pause any agent or workflow instantly without a code change.
- **Labour law:** statutory obligations (notice periods, final settlement timelines, PF, gratuity, leave rules) are encoded as rules, not left to the model to remember.

Cost and model rule

You can use any model from any vendor, but you are judged on measured cost per completed workflow, so choose the cheapest model that does each job well. HR services is not a frontier problem. A design that sends routine work to a large frontier model will score poorly on cost.

- **Measure, don't estimate.** Report cost in INR per completed workflow from real runs on the synthetic dataset, and project a monthly run cost at TBO's volumes (TBO will share assumed headcount and transaction volumes).
- **Use the right tool for each step.** Rule-based steps (eligibility checks, date calculations, routing) belong in code. Using an LLM where a rule would do counts against you.
- **Route between models.** Small or open-weight models handle the default path. A larger model is used only when a measured confidence or complexity threshold calls for it, and you must justify that threshold.
- **Avoid lock-in.** Changing the model provider must be a configuration change, not a rewrite.
- **Count everything.** Include hosting, GPU and inference-provider costs, not only token prices. A self-hosted model is not free.

What TBO provides

Every team gets the same starter kit, so results can be compared fairly. No real employee or candidate data will be shared.

Resource	What it contains
Synthetic HR dataset	Fictional employees, org chart, candidates, leave balances and exits
Mock IT and Admin APIs	Stubbed ticketing, asset (laptop) allocation, access cards, email and directory, and payroll setup
Sample HR policies	Onboarding, leave, notice period, exit and final settlement rules for your agents to reason over

You can extend the mocks, but your demo must run on the provided interfaces so that judges can compare teams.

Rounds and deliverables

There are three rounds, and each one gates the next. Dates and team sizes will be confirmed by TBO.

Round	You submit	How teams advance
1. Concept and architecture	An architecture document of up to 6 pages covering the chosen service line, the agent design, the governance model, the model choice with an estimated cost per workflow, and the extension model. Add a 5-minute video walkthrough.	Shortlisted on soundness of design, governance and cost logic
2. Build	A working demo on the synthetic dataset, a code repository with a README and setup steps, measured cost and automation figures, and a developer guide on how to add a new workflow	Finalists chosen on working software and measured results
3. Finale	A live demo and Q&A with the TBO panel, plus a **live extension test**: the panel gives a new HR requirement on the day and your team adds it to the platform within a set time	Winners decided on the evaluation criteria below

The live extension test is the hardest proof of objective 2. If adding a workflow means rewriting your core, it will show.

Evaluation criteria

Governance and extensibility together carry 40% of the score, as much as automation and cost combined. A clever demo that an HR team could not safely run, or that TBO developers could not extend, will not win.

Criterion	Weight	What judges look for
Governance and compliance	20%	Reserved decisions enforced, policy as configuration, audit trail, data protection
Extensibility and architecture	20%	Clean extension model, passing the live extension test, clear developer guide
Automation depth	20%	Share of steps completed without a human, cross-team loops closed, safe failure handling
Cost efficiency	15%	Measured cost per workflow, sensible model routing, realistic monthly projection
Engineering quality	15%	Code quality, tests, security, no duplicate actions on retry
Experience and demo	10%	How easy it is for employees, managers and HR to use, and how clearly it is presented

Rules

- **Data:** use only the synthetic dataset and mock APIs. Never put real personal data into any tool or model.
- **Open source:** any library or model you use must have a licence that allows commercial use. List each one in your README.
- **AI coding assistants:** allowed. Say which ones you used and for what.
- **Individual contribution:** keep a clean commit history. Finalists will be asked individually about the parts they built.
- **Intellectual property:** ownership and licensing of submissions will be set out in the participation terms. *(TBO to confirm before launch.)*